

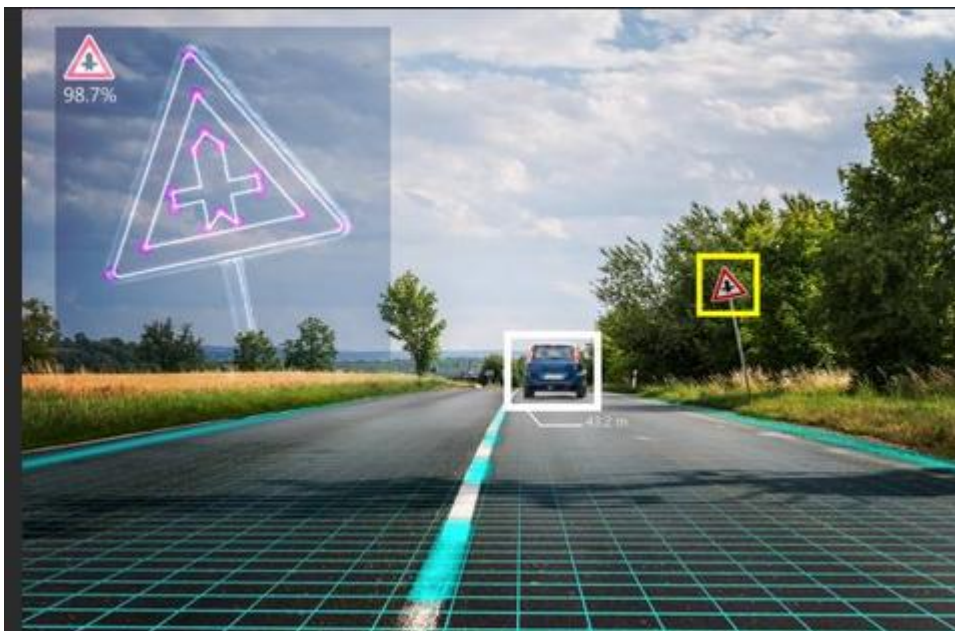
# Wykład 4

Splotowe Sieci Neuronowe (CNN)

Zastosowanie uczenia głębokiego do wizji komputerowej

# Wizja komputerowa

Wizja komputerowa to dziedzina sztucznej inteligencji, która umożliwia komputerom analizowanie, interpretowanie i rozumienie obrazów oraz filmów w sposób zbliżony do ludzkiej percepcji wzrokowej.



# Wizja komputerowa

Wizja komputerowa pozwala na automatyczne rozpoznawanie obiektów, śledzenie ruchu, analizowanie cech wizualnych i wyciąganie wniosków z danych wizualnych.

Wizja komputerowa znajduje szerokie zastosowanie w wielu branżach:

- przemysł motoryzacyjny (pojazdy autonomiczne),
- medycyna (analiza obrazów medycznych w celu diagnozowania chorób),
- nadzór,
- robotyka,
- handel detaliczny.

<https://przemyslprzyszlosci.gov.pl/technologie/cyfrowe/sztuczna-inteligencja/wizja-komputerowa/>

# Trochę historii

W 1998 roku konwolucyjne sieci neuronowe zostały zaprezentowane w artykule Bengio, Le Cun, Bottou i Haffnera.

Ich pierwsza konwolucyjna sieć neuronowa z 1994 r. nosiła nazwę LeNet-5 i była w stanie **klasyfikować cyfry z ręcznie zapisanych liczb**.

Właśnie ta sieć zapoczątkowała dziedzinę Deep Learning.

Sieć zawierała warstwy splotowe i warstwy łączące.

# Trochę historii

W latach 1998-2010 sieci neuronowe były w fazie inkubacji. Większość ludzi nie zauważyła ich rosnącej mocy, podczas gdy wielu innych badaczy powoli robiło postępy. Coraz więcej danych było dostępnych dzięki pojawieniu się aparatów w telefonach komórkowych i tanich aparatów cyfrowych. Moc obliczeniowa rosła, procesory stawały się coraz szybsze, a procesory graficzne stały się narzędziem obliczeniowym ogólnego przeznaczenia. Oba te trendy przyczyniły się do rozwoju sieci neuronowych, choć w powolnym tempie. Zarówno dane, jak i moc obliczeniowa sprawiały, że zadania, z którymi radziły sobie sieci neuronowe, stawały się coraz bardziej interesujące.

# Dan Ciresan Net

W 2010 roku Dan Claudiu Ciresan i Jurgen Schmidhuber opublikowali jedną z pierwszych implementacji sieci neuronowych na GPU. Implementacja ta obejmowała zaimplementowaną na procesorze graficznym **NVIDIA GTX 280 9-warstwową sieć neuronową**.

<https://dataconomy.com/2017/04/19/history-neural-networks/>

# AlexNet

W 2012 roku Alex Krizhevsky opublikował AlexNet, który był głębszą i znacznie szerszą wersją LeNet i wygrał z dużym marginesem trudny konkurs ImageNet.

Alex pracował w latach 2013–2017  
w AI Google w Mountain View, Kalifornia

Alex Krizhevsky, Alexei Krizhevsky  
(ukr. Олексій Крижевський,  
ur. 4 marca 1986 w Ukrainie),  
ukraińsko-kanadyjski informatyk



# AlexNet

AlexNet rozszerza LeNet do znacznie większej sieci neuronowej, która może być wykorzystana do uczenia się znacznie bardziej złożonych obiektów i hierarchii obiektów. Wkład tej pracy polegał na:

- wykorzystanie ReLU jako nieliniowości
- wykorzystanie techniki dropout do selektywnego ignorowania pojedynczych neuronów podczas uczenia, sposób na uniknięcie nadmiernego dopasowania modelu
- nakładające się maksymalne łączenie, unikające uśredniania efektów średniego łączenia
- wykorzystanie procesorów graficznych NVIDIA GTX 580 w celu skrócenia czasu uczenia

W tamtym czasie procesory graficzne oferowały znacznie większą liczbę rdzeni niż procesory centralne i umożliwiały 10-krotnie szybszy czas uczenia, co z kolei pozwalało na wykorzystanie większych zestawów danych, a także większych obrazów.

Sukces AlexNet zapoczątkował małą rewolucję. Konwolucyjne sieci neuronowe były teraz napędem Deep Learning, który stał się nową nazwą dla "dużych sieci neuronowych".

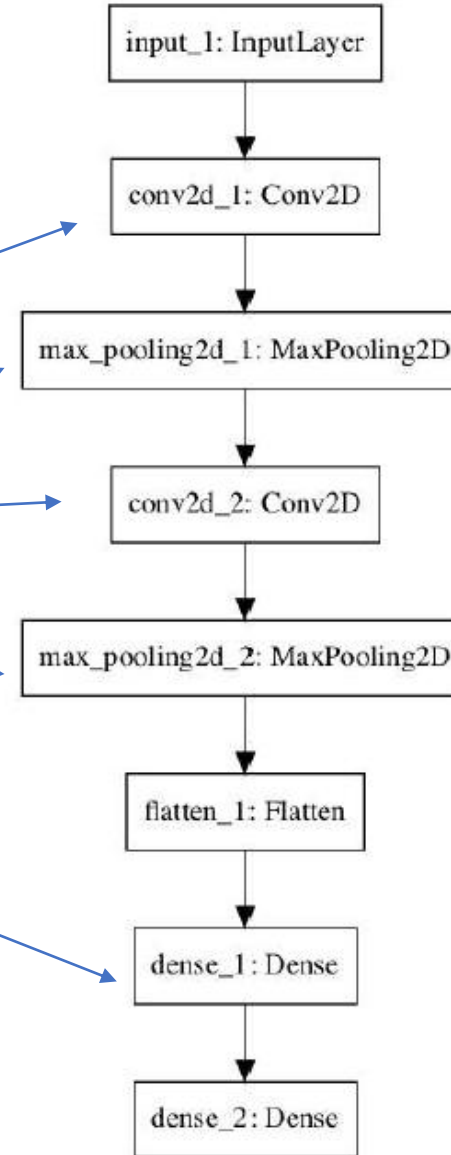


# Przykłady architektur konwolucyjnych CNN

- LeNet (1998),
- AlexNet (2012),
- GoogLeNet (2014),
- VGGNet (2014),
- ResNet (2015),

CNN składa się z trzech głównych warstw:

- Warstwy konwolucyjne
- Warstwy łączące/skalujące
- W pełni połączona warstwa gęsta



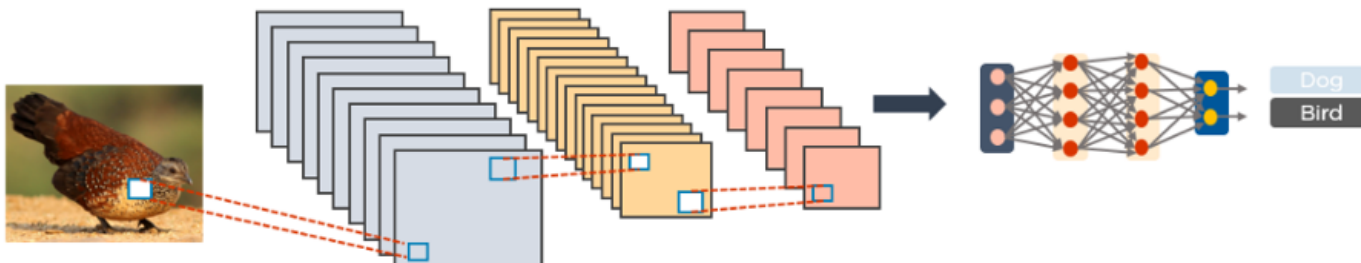
# Etapy działania sieci splotowej

Splot (convolucja) filtra/jądra z łatami obrazu wejścia+ReLU

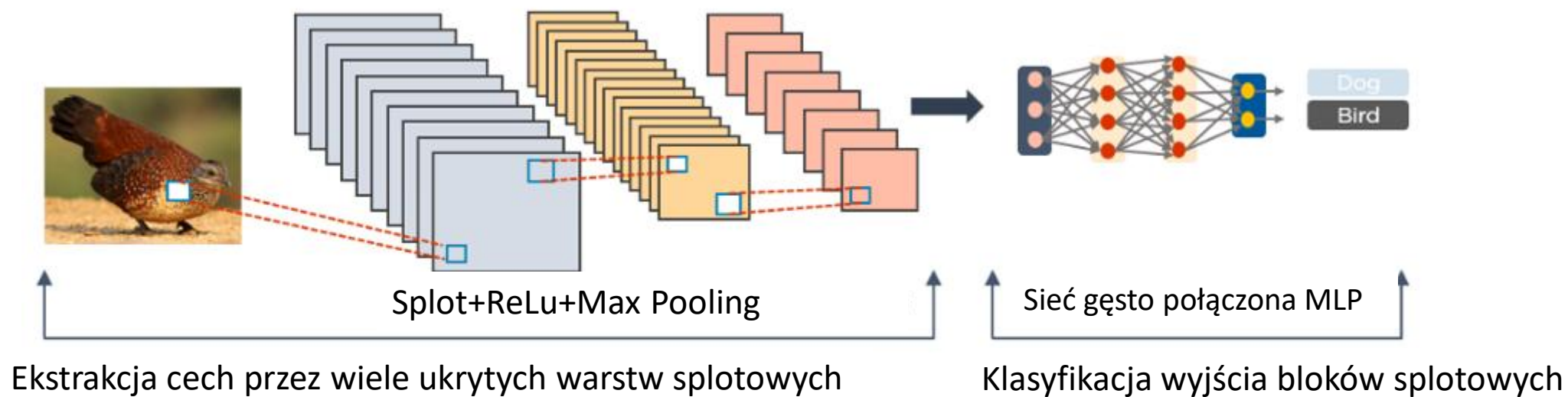
→mapa cech/ekstrakcja cech.

Zastosowanie filtra (okien ekstrakcji) do mapy cech w celu zmniejszenia wymiarowości (max-pooling)

→input jako wejście sieci gęstej.



# Przykład przetwarzania obrazu przez CNN



# Warstwa wejściowa

Reprezentuje ona obraz wejściowy do sieci w postaci tensorów.

W przypadku kolorowych zdjęć będziemy używać trzech kanałów wejściowych RGB, odpowiadającym odpowiednio kanałom czerwonym, zielonym i niebieskim.

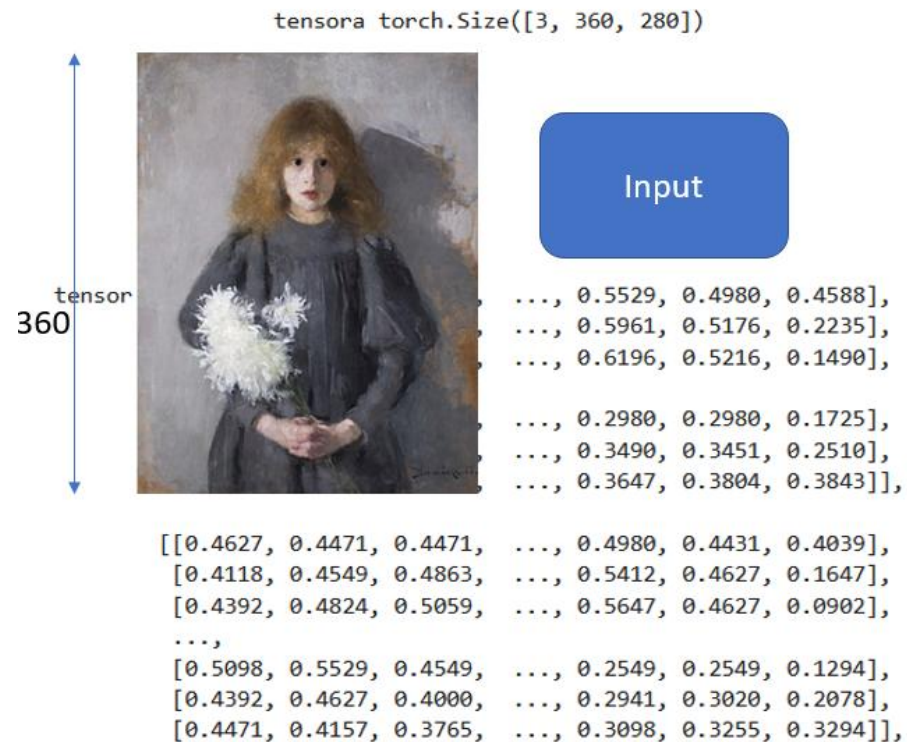
```
torch.utils.data.DataLoader
```

```
from torch.utils.data import DataLoader

# Konfiguracja hiperparametru wielkości paczki (batch)
BATCH_SIZE = 32

# Przekształcanie zbiorów danych w iterable (partie, paczki)
train_dataloader = DataLoader(train_data, # przekształca zbiór na iterowalne,
                              batch_size=BATCH_SIZE, # liczba paczek/partii danych
                              shuffle=True # Czy tasować dane co epokę?
                              )

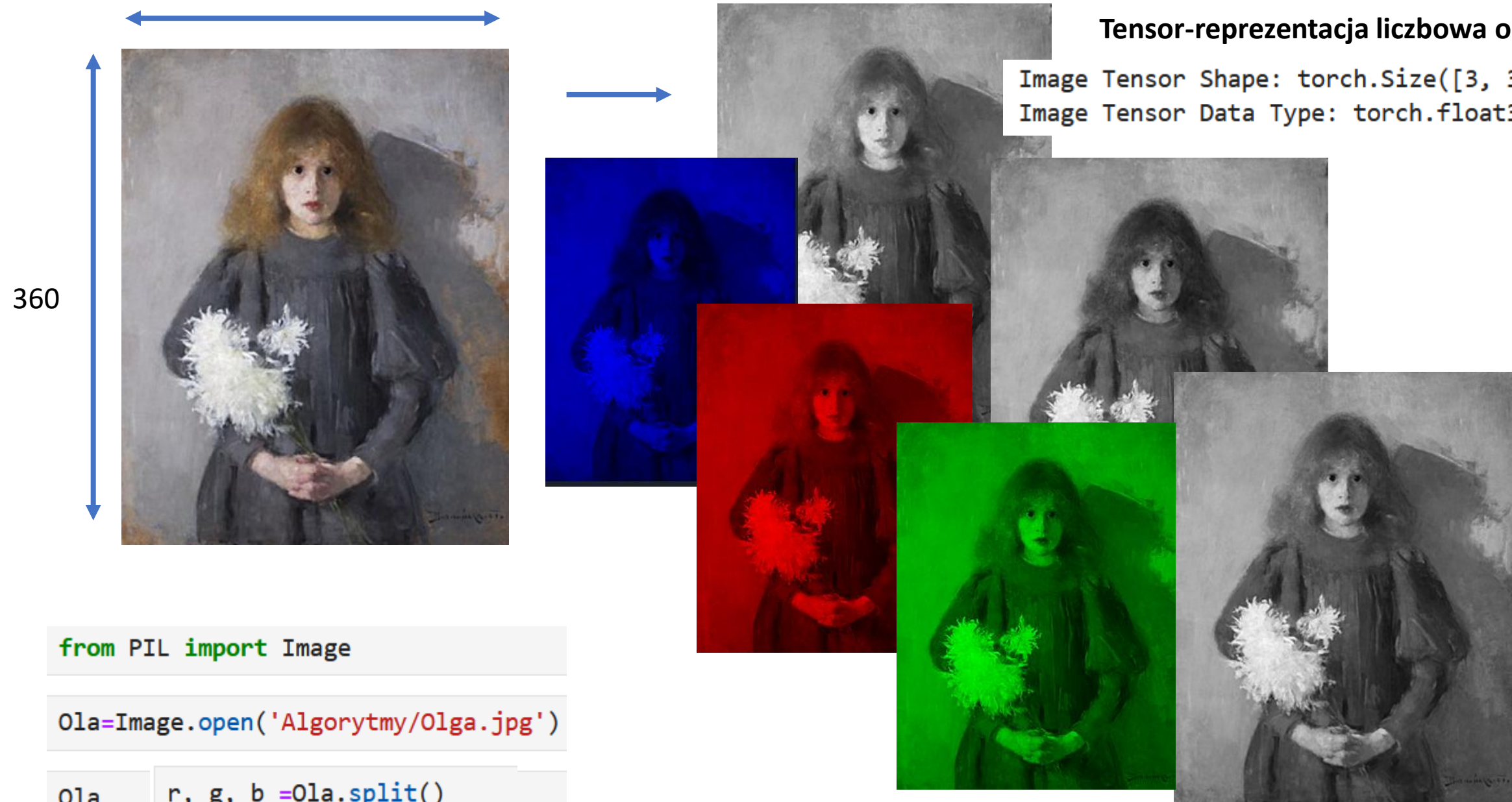
test_dataloader = DataLoader(test_data,
                              batch_size=BATCH_SIZE,
                              shuffle=False # nie jest konieczne tasowanie danych testowych
                              )
```



PIL (Python Imaging Library)

## Tensor-reprezentacja liczbowa obrazu

Image Tensor Shape: `torch.Size([3, 360, 280])`  
Image Tensor Data Type: `torch.float32`

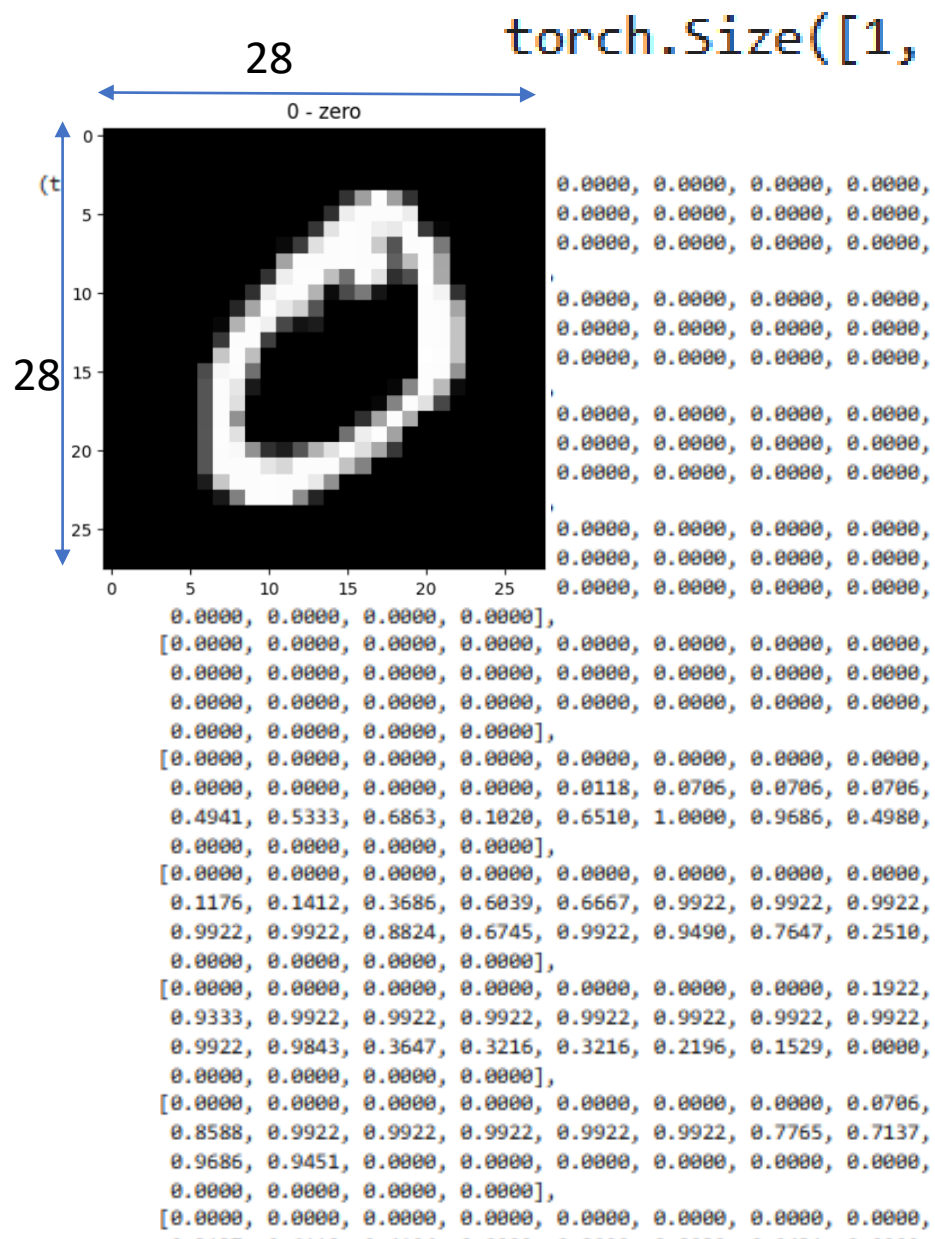


```
from PIL import Image
```

```
Ola=Image.open('Algorytmy/Olga.jpg')
```

```
Ola    r, g, b =Ola.split()
```





# Input

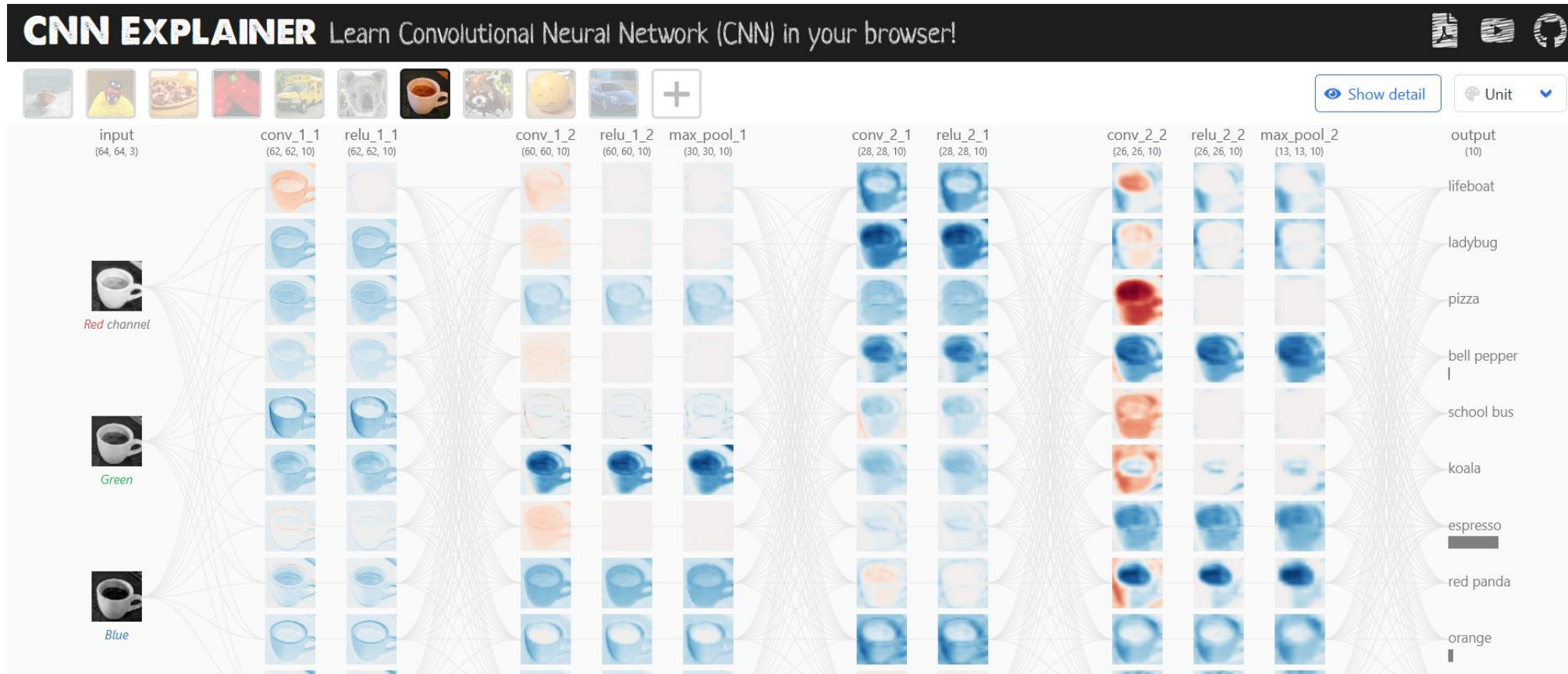
Obraz  
reprezentowany  
przez tensor

**W przypadku MNIST mamy skalę szarości i dlatego mamy jeden kanał.**

$$kształt(shape) = [None, 28, 28, 1], \quad NHWC$$
$$kształt(shape) = [None, 1, 28, 28], \quad NCHW$$
$$k\text{szta}\acute{t}(shape) = [32, 1, 28, 28], \quad NCHW$$

Batch size (rozmiar paczki danych)

- <https://bfirst.tech/konwolucyjne-sieci-neuronowe/>
- <https://poloclub.github.io/cnn-explainer/>
- <https://cs231n.github.io/convolutional-networks/>
- [2D Visualization of a Convolutional Neural Network](#)



Przykład działania sieci w Excelu: Wykład4\_CNN



# Filtry/Jądra

Filtry są podstawą działania sieci CNN. Są one podstawowym komponentem, który pomaga modelowi wyodrębnić użyteczne wzorce z danych wejściowych.

Rozmiar: Filtry są zazwyczaj małymi, kwadratowymi macierzami. Rozmiar filtra jest kluczowym hiperparametrem, który należy ustawić. Typowe wymiary to 3x3, 5x5 i 7x7.

Przesuwanie filtra: Filtr przesuwają się po danych wejściowych, przesuwając się za każdym razem o określoną liczbę pikseli, zdefiniowaną przez "krok" (stride 1 lub 2). W każdej pozycji filtr obejmuje niewielką część obrazu wejściowego.

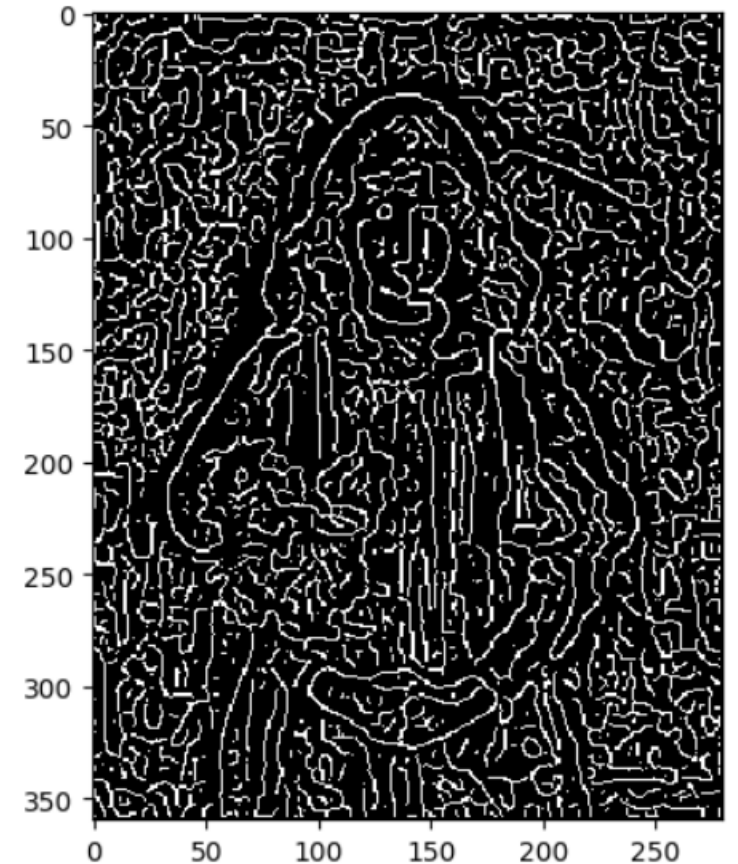
# Warstwa splotowa (convolutional layer)

Zawierają wyuczone filtry (jądra), które **wyodrębniają cechy odróżniające od siebie różne obrazy.**

**Wartości (wagi) w filtrach są dobierane i optymalizowane podczas uczenia sieci.**

Filtry są współdzielone na całym zdjęciu tzn. wagi dobierane są do filtra, który jest następnie przesuwany po całym zdjęciu.

Rozmiar samego zdjęcia nie wpływa na liczbę wag w warstwie konwolucyjnej.



# Splot

Jedna z funkcji splotu to dwuwymiarowa macierz zawierająca wartości pikseli obrazu (zazwyczaj o dużych wymiarach), a druga z funkcji to **filtr** (zdecydowanie mniejsza macierz), nazywana **jądrem konwolucji**.

W wyniku konwolucji obrazu z filtrem otrzymujemy nowy obraz.

```
nn.Conv2d(in_channels=input_shape,
          out_channels=hidden_units,
          kernel_size=3, # rozmiar filtra(jądra)
          stride=1, # krok domyślny
          padding=1), # opcje = „valid” (bez wypełnienia) lub „same” (wyjście ma taki sam kształt jak wejście) lub int dla określonej liczby
```

```
(block_1): Sequential(
  (0): Conv2d(1, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU()
  (2): Conv2d(10, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU()
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
```

# Filtr/Jądro

- Większość procesów przetwarzania obrazu musi działać na obrazach RGB z trzema kanałami. Obraz RGB jest trójwymiarową tablicą liczb, znaną również jako tensor trzeciego rzędu.
- Podczas przetwarzania trzykanałowego obrazu RGB zwykle używane jest jądro konwolucyjne, które jest trójwymiarową tablicą / tensorem liczb o wymiarze 3. Bardzo często jądro konwolucyjne ma rozmiar  $3 \times 3 \times 3$  - jądro konwolucyjne przypomina sześcian.

# Operacja splotu

Uczenie się: Podczas **uczenia CNN** uczy się **najlepszych wartości dla wag filtrów**, aby zrealizować dane zadanie (np. klasyfikowanie obrazów, wykrywanie obiektów itp.). Odbywa się to za pomocą algorytmów wstecznej propagacji i optymalizacji, podobnie jak w przypadku innych wag w sieci neuronowej.

**Ekstrakcja cech:** Filtry są odpowiedzialne za ekstrakcję cech w CNN.

Na przykład niektóre filtry mogą specjalizować się w wykrywaniu poziomych krawędzi na obrazie, inne mogą wykrywać pionowe krawędzie, kolory, tekstury itp. Gdy model staje się głębszy, filtry mogą rozpoznawać bardziej złożone wzorce.

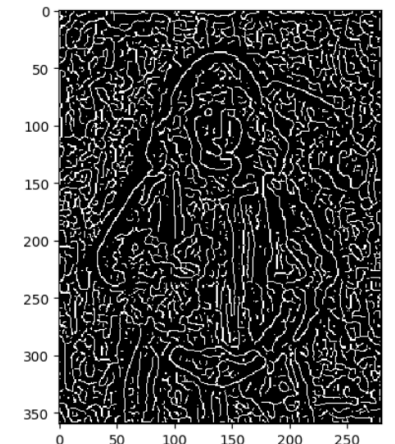
Stosując wiele filtrów do danych wejściowych, CNN może nauczyć się wykrywać szeroką gamę funkcji. To czyni je potężnymi narzędziami do zadań przetwarzania obrazu.

# Splot

Zadaniem warstwy konwolucyjnej, w przypadku pierwszej warstwy, jest **wyodrębnienie cech, takich jak krawędzie, kolory, gradienty**.

Kolejne warstwy sieci – korzystając z tego, co wyznaczyły poprzednie warstwy – mogą wykrywać coraz bardziej skomplikowane kształty.

Analogicznie do warstw zwykłej sieci, po warstwie konwolucyjnej występuje warstwa aktywacyjna (najczęściej funkcja ReLU), wprowadzająca nieliniowość do sieci.



# Hiperparametry warstwy splotu `nn.Conv2d()`

Do pełnego zdefiniowania warstwy splotowej musimy określić:

- Kernel Size (rozmiar jądra/rozmiar filtra)
  - Padding (wypełnienie)
  - Stride (krok)- krok przesunięcia okna filtra
  - Funkcje aktywacji
- `in_channels` (int) - Liczba kanałów w obrazie wejściowym.
  - `out_channels` (int) - Liczba kanałów utworzonych przez konwolucję.
  - `kernel_size` (int lub tuple) - Rozmiar jądra konwolucji/filtra.
  - `stride` (int lub krotka, opcjonalnie) - Jak duży krok wykonuje jądro konwolucji na raz. Domyślnie: 1.
  - `padding` (int, tuple, str) - Wypełnienie dodane do wszystkich czterech stron wejścia. Domyślnie: 0.



# Przykład architektury sieci splotowej

```
def __init__(self, input_shape: int, hidden_units: int, output_shape: int):  
    super().__init__()  
    self.block_1 = nn.Sequential(  
        nn.Conv2d(in_channels=input_shape,  
                  out_channels=hidden_units,  
                  kernel_size=3, # rozmiar filtra(jądra)  
                  stride=1, # krok domyślny  
                  padding=1), # opcje = „valid” (bez wypełnienia) lub „same” (wyjście ma taki sam kształt jak wejście) lub int dla określonej liczby  
        nn.ReLU(),  
        nn.Conv2d(in_channels=hidden_units,  
                  out_channels=hidden_units,  
                  kernel_size=3,  
                  stride=1,  
                  padding=1),  
        nn.ReLU(),  
        nn.MaxPool2d(kernel_size=2,  
                     stride=2) # domyślna wartość kroku jest taka sama jak kernel_size
```

Warstwa wejściowa -> [Warstwa splotowa -> warstwa aktywacji -> warstwa łączenia] -> Warstwa wyjściowa

Blok, który może się powtarzać

# Przykład architektury sieci splotowej

```
def __init__(self, input_shape: int, hidden_units: int, output_shape: int):
    super().__init__()
    self.block_1 = nn.Sequential(
        nn.Conv2d(in_channels=input_shape,
                  out_channels=hidden_units,
                  kernel_size=3, # rozmiar filtra(jądra)
                  stride=1, # krok domyślny
                  padding=1), # opcje = „valid” (bez wypełnienia) lub „same” (wyjście ma taki sam kształt jak wejście) lub int dla określonej liczby
        nn.ReLU(),
        nn.Conv2d(in_channels=hidden_units,
                  out_channels=hidden_units,
                  kernel_size=3,
                  stride=1,
                  padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2,
                     stride=2) # domyślna wartość kroku jest taka sama jak kernel_size
```

- `nn.Conv2d()` , warstwa splotowa.
- `nn.MaxPool2d()` , warstwa maksymalnego łączenia.

2d oznacza dane dwuwymiarowe (obrazy mają 2 wymiary, wysokość i szerokość).

W przypadku innych danych wymiarowych (takich jak 1D dla tekstu lub 3D dla obiektów 3D) istnieją również `nn.Conv1d()` i `nn.Conv3d()`.

# Warstwa łącząca (Pooling)

**Skalowanie/łączenie w konwolucyjnych** sieciach neuronowych (CNN) jest podstawową techniką stosowaną do zmniejszania wymiarów przestrzennych map cech przy jednoczesnym zachowaniu ważnych informacji z obrazu wejściowego.

Ogólnie mówiąc, używamy warstw łączenia, gdy chcemy wykryć obiekt na obrazie, niezależnie od jego lokalizacji na obrazie.

Warstwy łączące w CNN są zwykle wstawiane między warstwy konwolucyjne i służą jako środek podsumowujący, a także konsolidujący mapy funkcji utworzone przez te warstwy.

Obecność warstw łączenia w CNN pozwala nam stopniowo zmniejszać wymiary przestrzenne, aby zminimalizować liczbę parametrów.

# Max pooling/average pooling `nn.MaxPool2d()`

Operacja max-pooling jest przeprowadzana przy użyciu okien ekstrakcji przetwarzających mapy cech.

Innymi słowy stosujemy filtr tym razem dla mapy cech, łaty nie nakładają się.

Zwykle używamy okien 2x2 i kroku 2.

Okna ekstrakcji zwracają maksymalne wartości każdego z kanałów.

Operacja wybiera miejsca, gdzie filtr wyjściowy „dobrze” dokonał zadanie.

`nn.AvgPool2d()`

# Padding/Dopełnianie

Padding to kolejne ważne pojęcie używane w kontekście konwolucyjnych sieci neuronowych. Odnosi się ono do procesu dodawania dodatkowych warstw pikseli wokół granicy obrazu.

Kiedy wykonujemy operację splotu, przesuwamy filtr po obrazie. Jeśli przesuniemy filtr tylko po oryginalnych pikselach obrazu, wynikowy wynik będzie mniejszy niż obraz wejściowy. Na przykład, jeśli mamy obraz  $5 \times 5$  (25 pikseli) i używamy filtra  $3 \times 3$ , wynikowym wynikiem będzie macierz  $3 \times 3$  (9 pikseli), ponieważ filtr  $3 \times 3$  może zmieścić się w obrazie  $5 \times 5$  tylko w dziewięciu różnych pozycjach.

W niektórych przypadkach korzystne jest zachowanie wymiarów przestrzennych (tj. szerokości i wysokości) danych wyjściowych takich samych jak danych wejściowych. W tym miejscu pojawia się padding. Dodając warstwę zer wokół granicy obrazu, możemy zastosować filtr do większej liczby pozycji, skutecznie zachowując wymiary przestrzenne wyjścia.

# Padding/Dopełnianie

Istnieją dwa główne rodzaje wypełnienia:

**Valid Padding** (Bez wypełnienia): W tym przypadku filtr jest stosowany tylko do prawidłowych pozycji wewnątrz obrazu, nie wykraczając poza granicę. Skutkuje to mniejszymi wymiarami wyjściowymi.

**Same Padding**: Obraz jest wypełniany wystarczającą liczbą zer wokół granicy, aby wymiary wyjściowe po operacji splotu były takie same jak wymiary wejściowe.

Wypełnienie służy kilku celom:

Zachowanie wymiarów: padding może pomóc w utrzymaniu wymiarów przestrzennych danych wejściowych przez warstwy sieci.

Informacje graniczne: Padding pozwala sieci na uwzględnienie informacji na granicach obrazu.

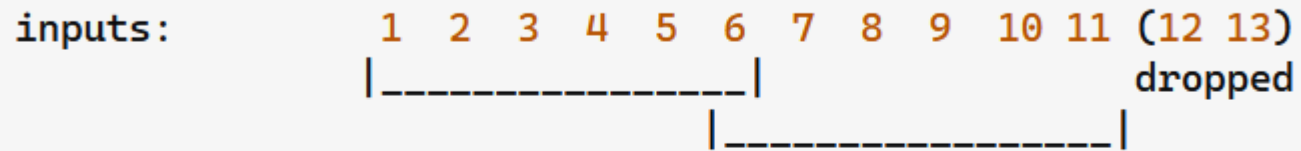
Bez paddingu filtry byłyby stosowane głównie do centralnych pikseli obrazu, co mogłoby prowadzić do utraty informacji z krawędzi obrazu.

Podobnie jak w przypadku kroków, wybór typu wypełnienia jest hiperparametrem sieci i może wymagać dostrojenia podczas procesu uczenia modelu.

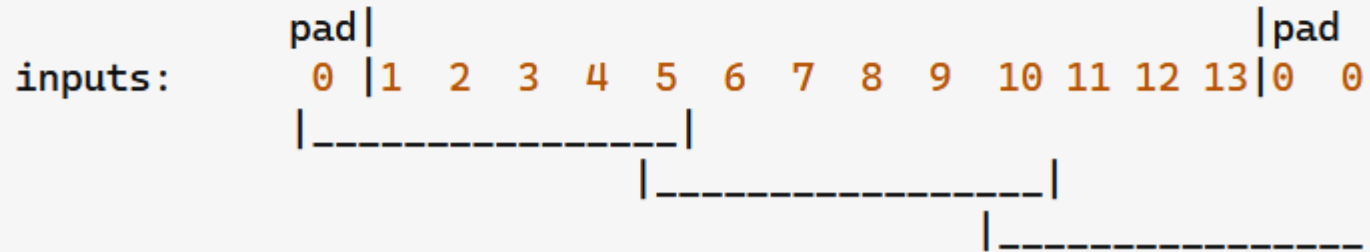
```
padding=0) # spróbuj także "valid" lub "same"
```

# Padding

- "VALID" = without padding:



- "SAME" = with zero padding:



- <https://www.geeksforgeeks.org/cnn-introduction-to-padding/>

## Działanie bloku

tensorA:

```
tensor([[[[0, 0, 1, 1, 0, 0],
          [0, 1, 0, 0, 1, 0],
          [1, 0, 0, 0, 0, 1],
          [1, 0, 0, 0, 0, 1],
          [0, 1, 0, 0, 1, 0],
          [0, 0, 1, 1, 0, 0]]]])
```

Kształt TensorA: torch.Size([1, 1, 6, 6])

Typ danych: torch.int64

TesnorA po max\_pool

```
tensor([[[[1, 1, 1, 1, 1],
          [1, 1, 0, 1, 1],
          [1, 0, 0, 0, 1],
          [1, 1, 0, 1, 1],
          [1, 1, 1, 1, 1]]]]) <- jest to maksymalna wart
```

Kształt tensoraA po max\_pool: torch.Size([1, 1, 5, 5])

```
conv_layer = nn.Conv2d(in_channels=1,
                        out_channels=1,
                        kernel_size=3,
                        stride=1,
                        padding=0)

# Przekazanie danych przez warstwę konwolucyjną
conv_layer(tensorA.float())
```

```
tensor([[[[-0.4618, -0.2953, -0.2786, -0.4076],
          [-0.1938, -0.2540, -0.3305, -0.2760],
          [-0.1902, -0.2290, -0.2514, -0.3266],
          [-0.4076, -0.1876, -0.2251, -0.4618]]]],
        grad_fn=<ConvolutionBackward0>)
```

```
[12] max_pool_tensor = max_pool_layer(conv_layer(tensorA.float()))
max_pool_tensor
```

```
tensor([[[[-0.1938, -0.2540, -0.2760],
          [-0.1902, -0.2290, -0.2514],
          [-0.1876, -0.1876, -0.2251]]]],
        grad_fn=<MaxPool2DWithIndicesBackward0>)
```



# Podsumowanie

Splotowe sieci neuronowe służą przede wszystkim do:

- Klasyfikacji obrazów;
- Rozpoznawania obrazów;

Cechy:

Zmniejszają liczbę neuronów wejścia;

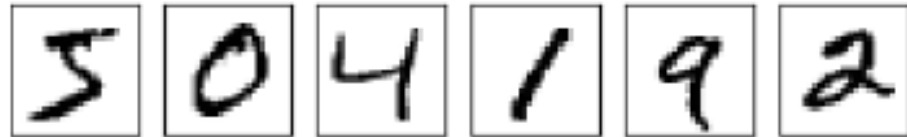
Są odporne na małe przesunięcia/rotacje;

Uwzględniają podobieństwa fragmentów obrazów.

Każda warstwa w sieci splotowej próbuje skompresować dane z przestrzeni o wyższym wymiarze do przestrzeni o niższym wymiarze.

# Przykład ćwiczenia 6

Baza MNIST



Warstwa wejściowa sieci zawiera neurony kodujące wartości pikseli wejściowych. Dane treningowe dla sieci będą składać się z wielu obrazów 28 na 28 pikseli zeskanowanych cyfr pisma odręcznego, a zatem warstwa wejściowa zawiera  $784 = 28 \cdot 28$  neuronów.

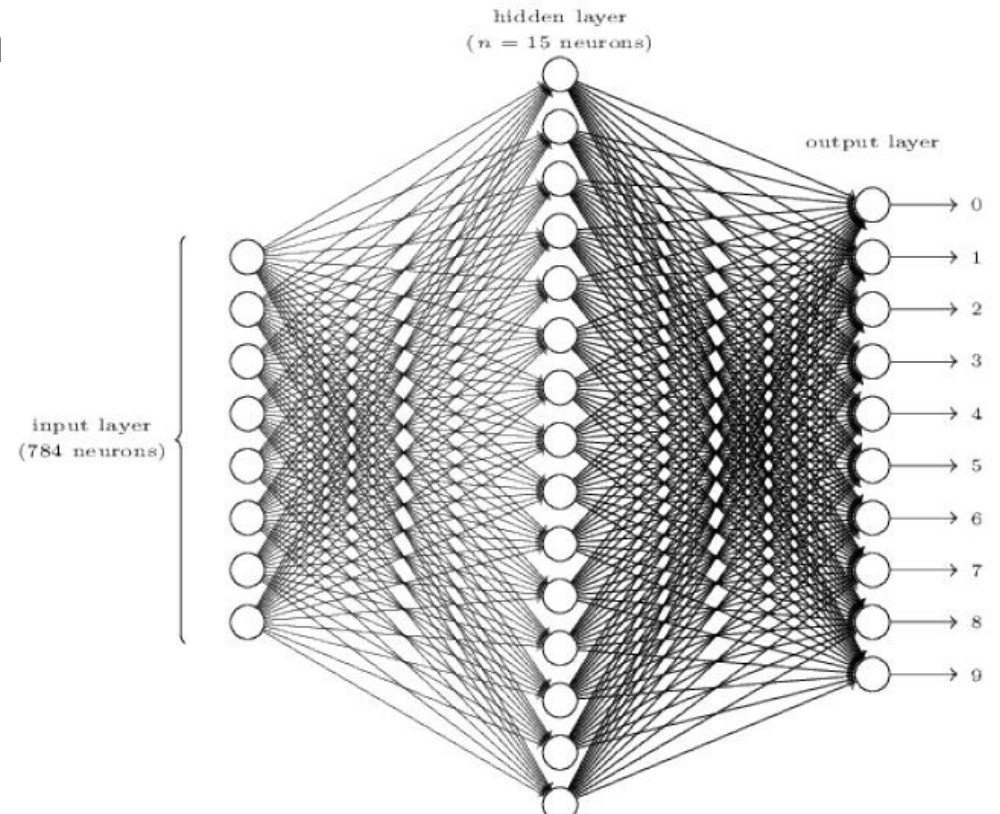
Piksele wejściowe są w skali szarości, z wartością 0,0 reprezentującą biel, wartością 1,0 reprezentującą czerń, a wartości pomiędzy nimi reprezentują stopniowo ciemniejące odcienie szarości.

Drugą warstwą sieci jest warstwa ukryta. Liczbę neuronów oznaczamy jako  $n$  (do eksperymentalnego wyznaczenia), architektura MLP;

Warstwa wyjściowa sieci zawiera 10 neuronów. Jeśli pierwszy neuron uruchomi się, tj. będzie miał na wyjściu 1, będzie to oznaczać, że sieć uważa, że cyfra to 0.

Jeśli drugi neuron zadziała, będzie to ozn  
że cyfra to 1.

Dokładniej,  
numerujemy neurony wyjściowe  
od 0 do 9 i  
ustalamy, który neuron ma najwyższą  
wartość aktywacji.



# Przykład ćwiczenia 5 i 6

Porównanie modelu MLP z modelem CNN